

# Resumen de Estrategias de Automatización QA

## 1. Automatización Frontend: Sauce Demo E2E

Para este reto, el objetivo fue validar el flujo completo de compra en la aplicación Sauce Demo. Utilicé un stack moderno basado en **Playwright** y **Cucumber**.

- **Cobertura:** Se automatizaron 8 escenarios que cubren desde el inicio de sesión (casos positivos y negativos) hasta la confirmación final de la compra.
- **Diseño:** Implementé el patrón **Page Object Model (POM)** para separar la lógica de las pruebas de los selectores de la interfaz, lo que facilita el mantenimiento futuro.
- **Legibilidad:** Las pruebas están escritas en **Gherkin (español)**, lo que permite que cualquier miembro del equipo entienda qué se está probando sin necesidad de leer código técnico.
- **Robustez:** Se configuró un sistema de captura automática de evidencia (screenshots) que se activa ante cualquier fallo, facilitando la depuración.

## 2. Automatización Backend: API ServeRest

Este proyecto se centró en probar el ciclo de vida completo de los usuarios (CRUD) en la API pública ServeRest. Para ello, utilicé **Karate DSL** con **JUnit 5**.

- **Cobertura:** Se diseñaron 27 escenarios de prueba que abarcan métodos GET, POST, PUT y DELETE.
- **Independencia de datos:** Implementé un generador de datos dinámicos basado en timestamps para asegurar que cada ejecución sea independiente y repetible sin colisiones de datos.
- **Validación de Contratos:** Cada respuesta de la API se valida automáticamente contra un esquema JSON externo, asegurando que cualquier cambio inesperado en la estructura de los datos sea detectado de inmediato.
- **Eficiencia:** La suite completa se ejecuta en aproximadamente 30 segundos, proporcionando un feedback rápido sobre el estado de la API.

## **Conclusión y Repositorio**

Ambas soluciones han sido integradas en un único repositorio para facilitar su revisión.

[GxIman10/NTT-DATA\\_QA-Automation\\_Gelman-Chavez](#)